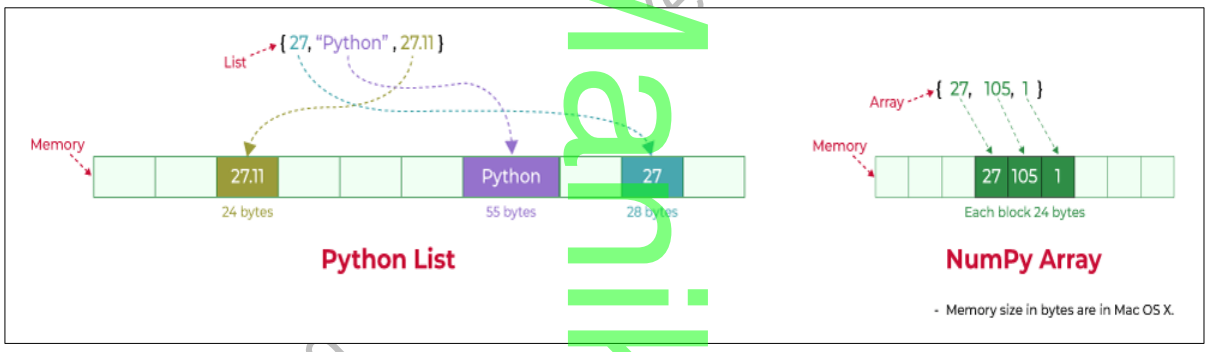
3. **Memory Fragmentation:** Lists may not store elements in contiguous memory locations, causing memory fragmentation and inefficiency.
4. **Performance:** Lists are not optimized for numerical computations and may have slower mathematical operations due to Python's interpretation overhead. They are generally used as general-purpose data structures.
5. **Functionality:** Lists can store any data type, but lack specialized NumPy functions for numerical operations.

Numpy Arrays

1. **Homogeneous Data:** NumPy arrays store elements of the same data type, making them more compact and memory-efficient than lists.
2. **Fixed Data Type:** NumPy arrays have a fixed data type, reducing memory overhead by eliminating the need to store type information for each element.
3. **Contiguous Memory:** NumPy arrays store elements in adjacent memory locations, reducing fragmentation and allowing for efficient access.
4. **Array Metadata:** NumPy arrays have extra metadata like shape, strides, and data type. However, this overhead is usually smaller than the per-element overhead in lists.
5. **Performance:** NumPy arrays are optimized for numerical computations, with efficient element-wise operations and mathematical functions. These operations are implemented in C, resulting in faster performance than equivalent operations on lists.

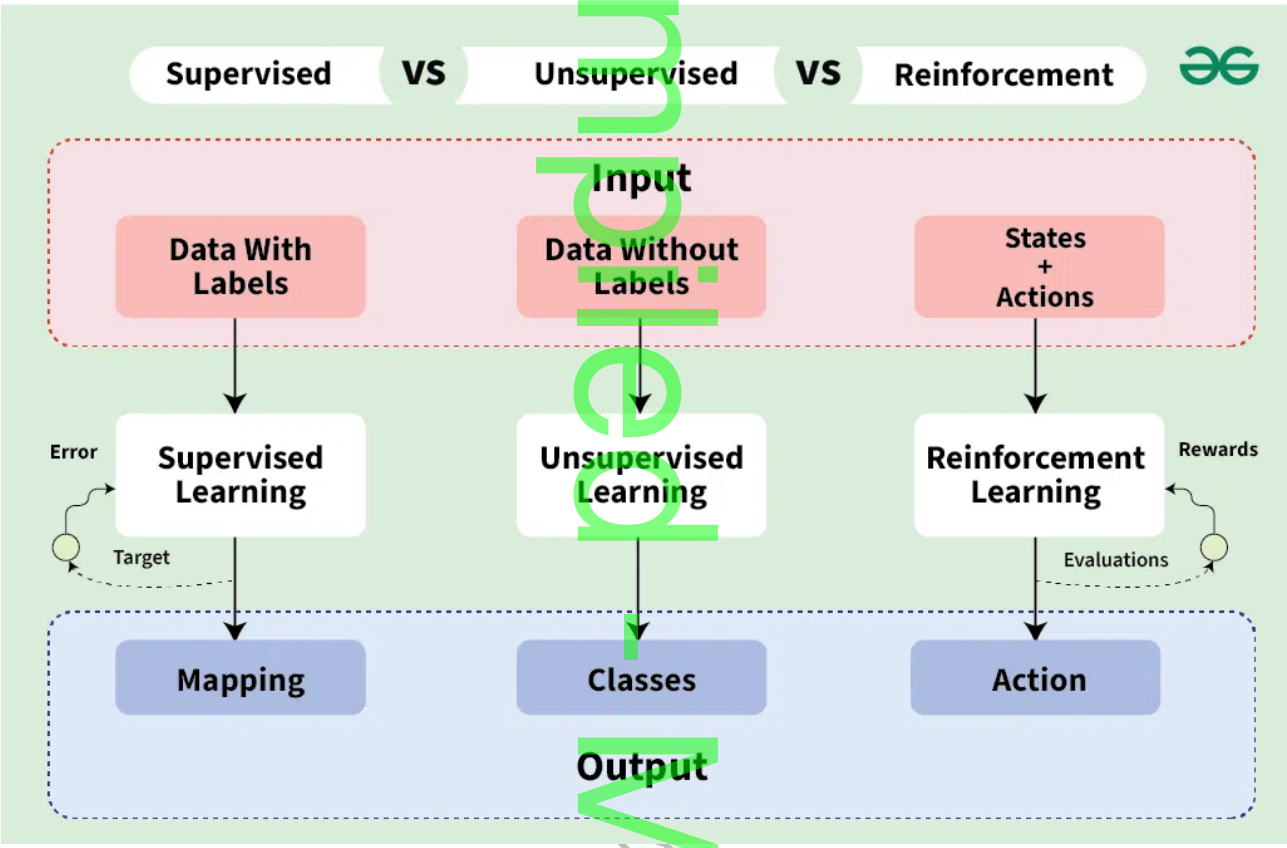
Comparison between Numpy array and Python List



Compiled by Manikandan R

UNIT -3

Supervised Unsuperivsed Reinforcement



1. Supervised Learning

Supervised learning is like learning with a teacher. The model is trained on a labeled dataset, meaning each input has a corresponding output. The key characteristics of supervised learning are:

- **Labeled Data:** Training data has predefined labels.
- **Types of Problems:** Used for classification task like spam detection and regression task like predicting house prices.
- **Algorithms:** Linear Regression, Logistic Regression, SVM, Decision Trees, Neural Networks.

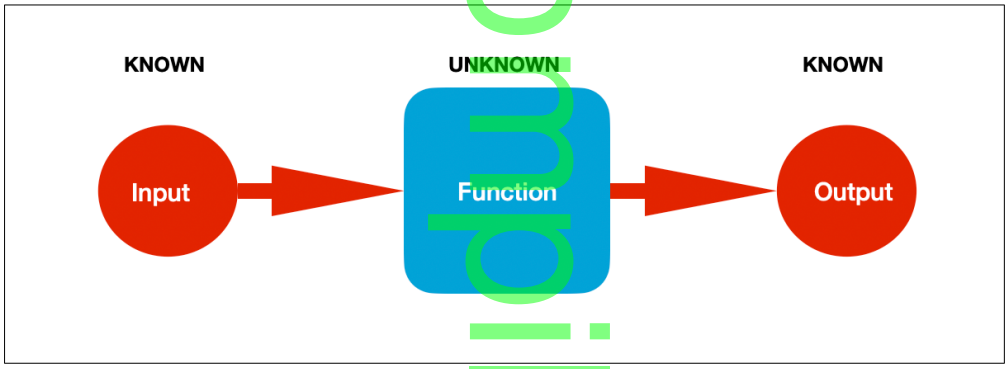
2. Unsupervised Learning

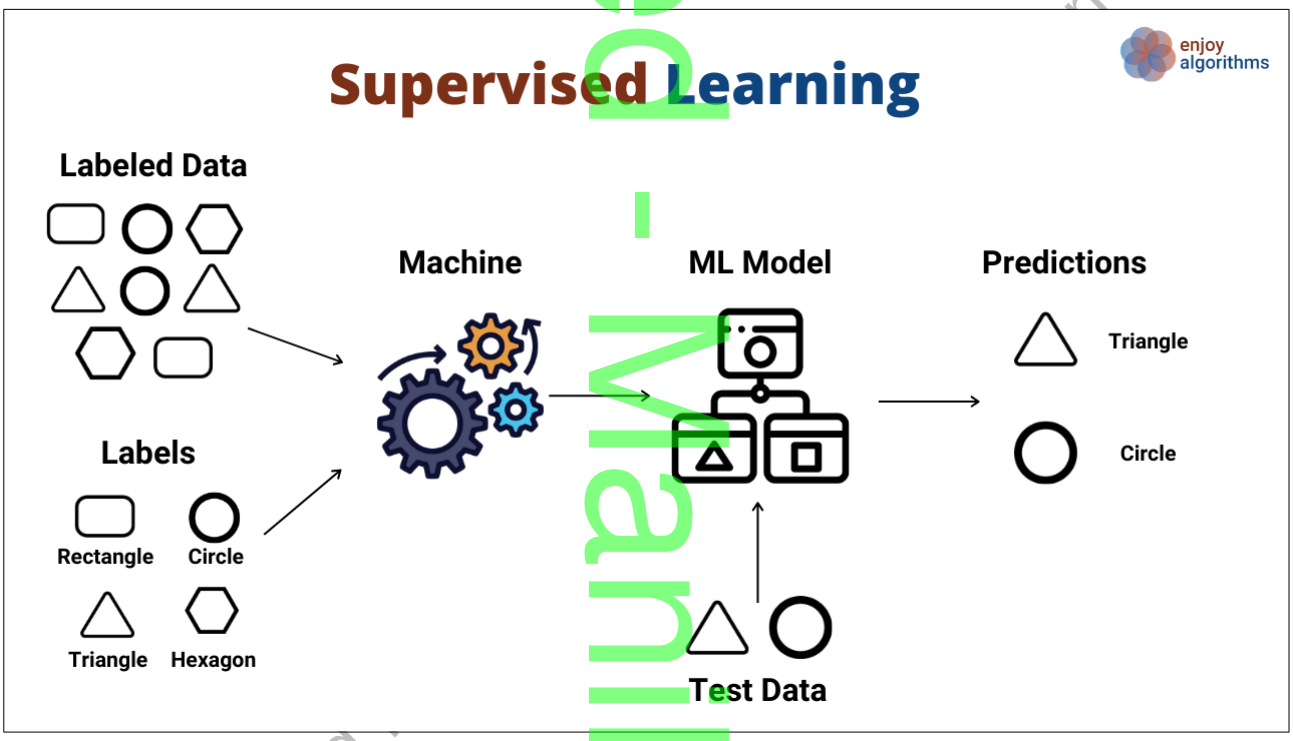
Unsupervised learning works with data that has no predefined labels. The model identifies patterns, clusters or associations independently. The key characteristics of unsupervised learning are:

- **Unlabeled Data:** No predefined outputs.
- **Types of Problems:** Used for Clustering task like customer segmentation and association task like market basket analysis.
- **Algorithms:** K-Means, Hierarchical Clustering, PCA, Autoencoders.

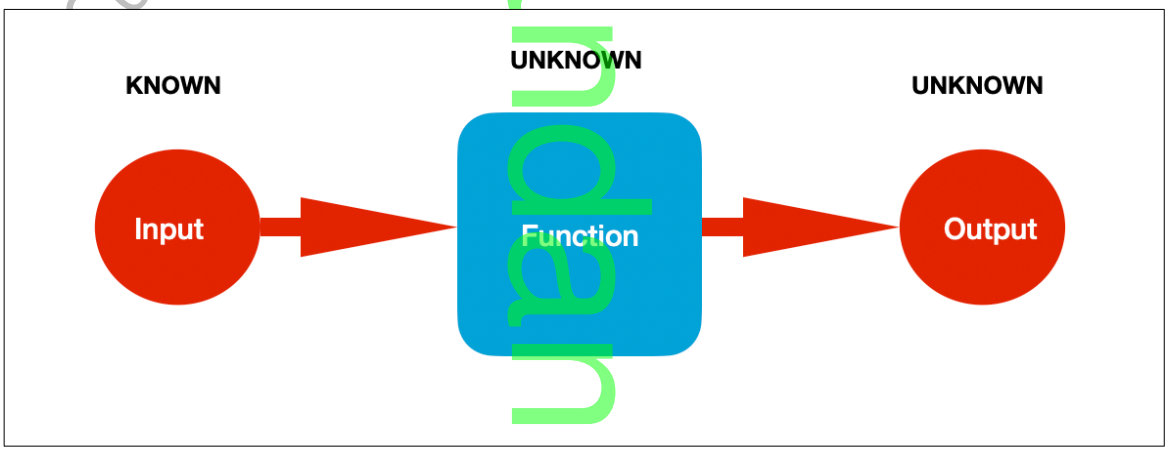
Compiled for EEE Final Year Students

Supervised Learning

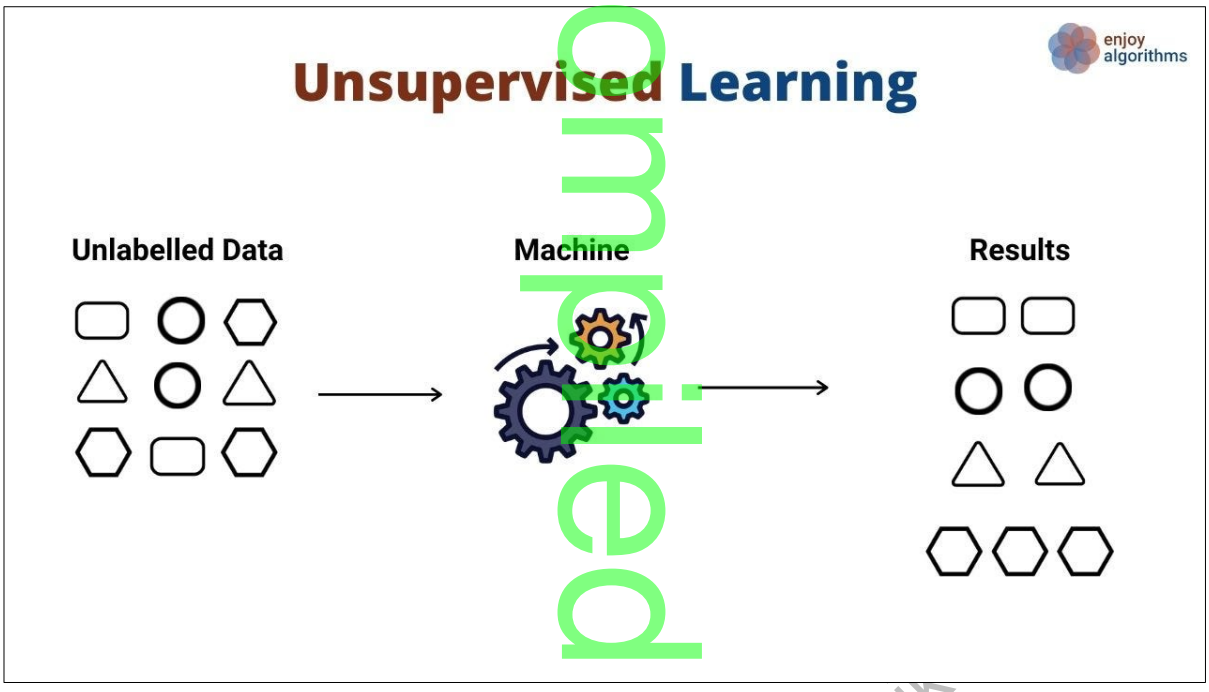




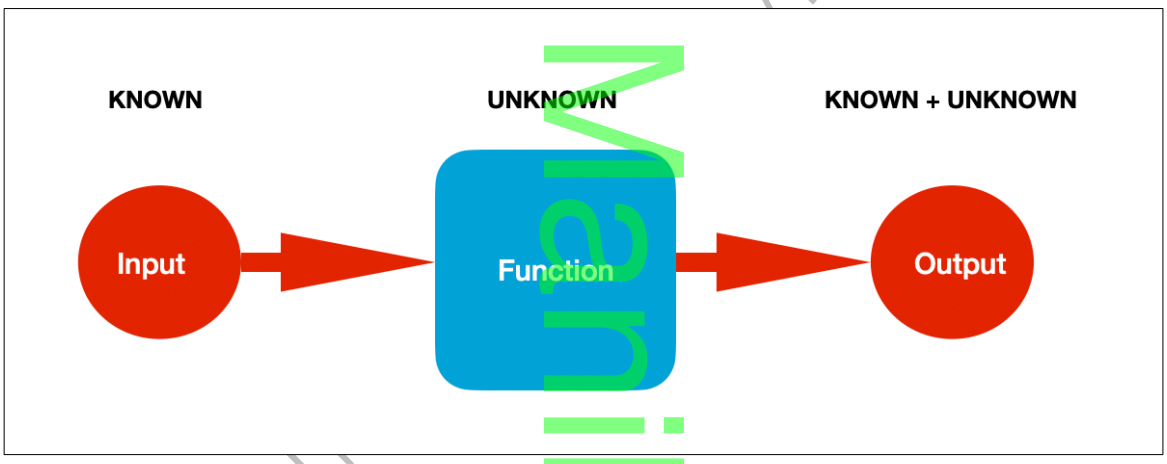
Unsupervised Learning

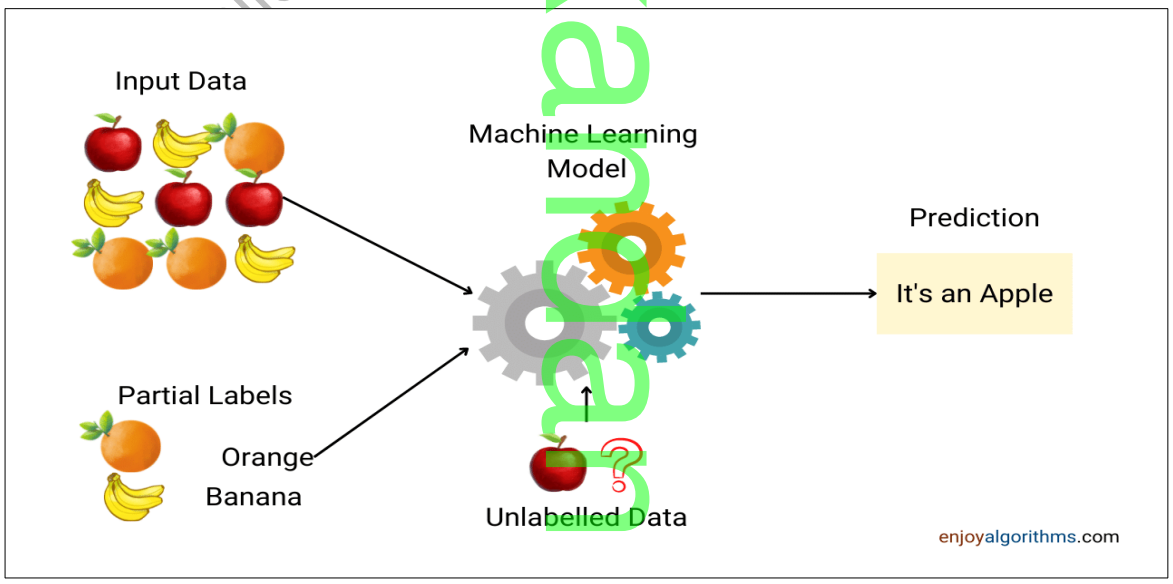


Compiled - Manikandan R

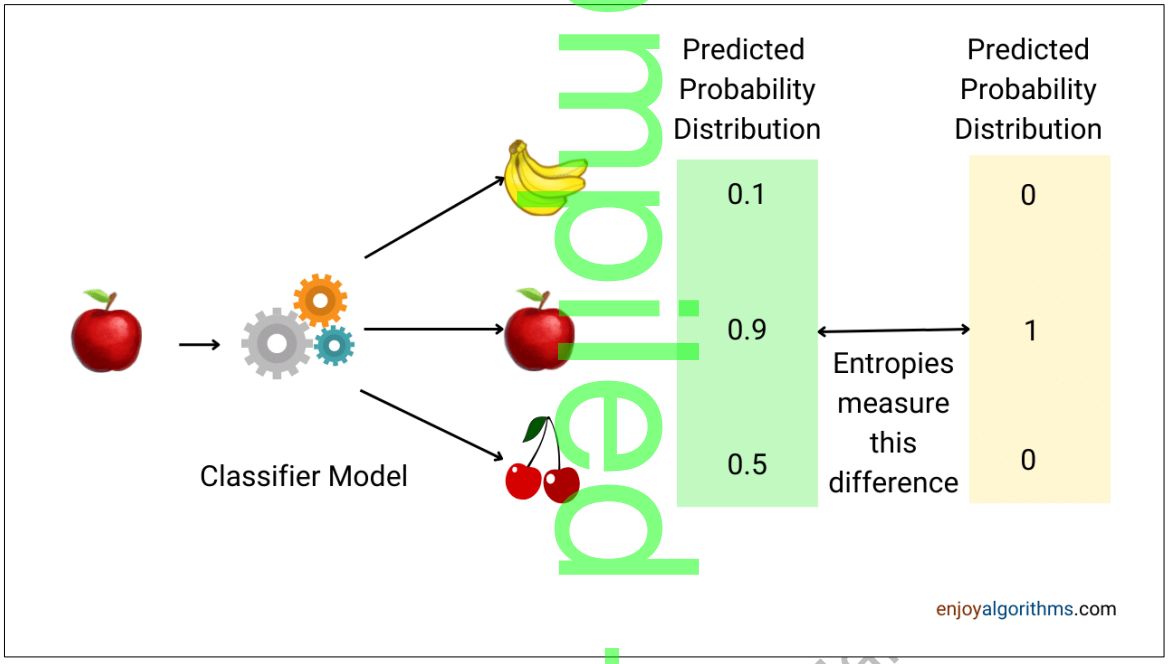


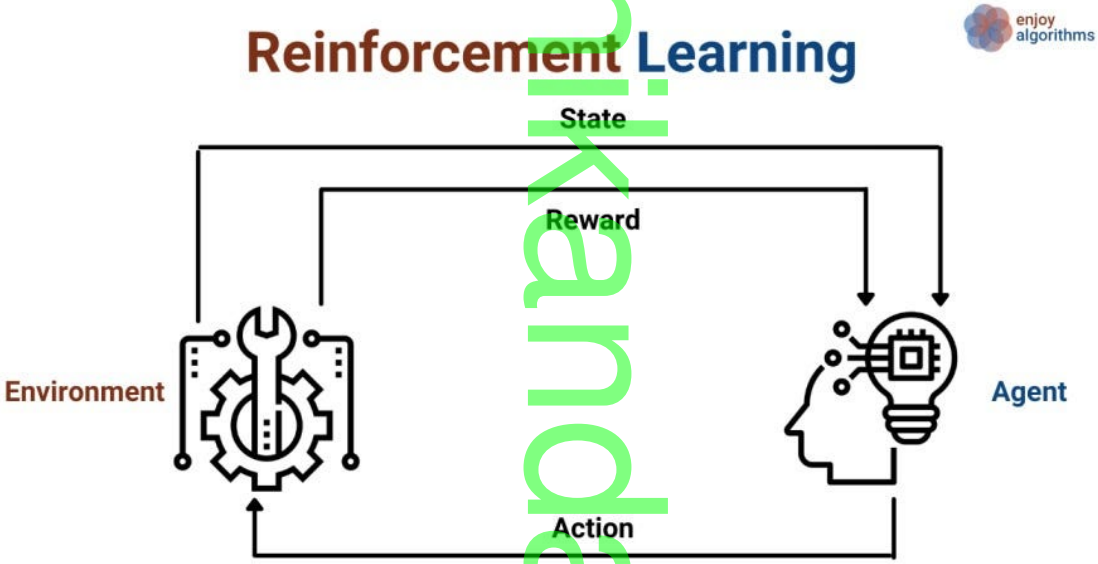
Semi Supervised Learning Algorithms





Compiled by Manikanta R





Compiled - Manikandan R
EE Final Year by Ma

Supervised Unsuperivsed & Semi-Supervised Machine Learning Algorithm

Type	Supervised	Unsupervised	Semi-supervised
Input Data	Labelled	Unlabelled	Partially Labelled
Computational Complexity	Simpler	Computationally Complex	Depends on use of Supervised or unsupervised
Accuracy	Higher	Lesser	Lesser

Differentiate Supervised, Unsupervised and Reinforrcement ML Model

Criteria	Supervised Learning	Unsupervised Learning	Reinforcement Learning
Definition	Learns from labeled data	Identifies patterns in unlabeled data	Learns through interaction with environment
Type of Data	Labeled data	Unlabeled data	No predefined data learn from environment
Type of Problems	Classification, Regression	Clustering, Association	Sequential decision-making
Supervision	Requires external supervision	No supervision	No supervision, learns from feedback
Algorithms	SVM, Decision Trees, Neural Networks	K-Means, PCA, Autoencoders	Q-learning, DQN, SARSA
Goal	Predict outcomes accurately	Discover hidden patterns	Optimize actions for maximum rewards
Applications	Medical diagnosis, fraud detection	Customer segmentation, anomaly detection	Self-driving cars, robotics, gaming

Real-World Applications

Machine Learning Type	Domain	Examples
Supervised Learning	Healthcare	Disease diagnosis like cancer detection
	Finance	Loan approval, credit risk assessment
	NLP	Sentiment analysis, text classification
Unsupervised Learning	E-commerce	Product recommendation, customer segmentation
	Cybersecurity	Fraud detection, intrusion detection
	Biology	Gene classification, dimensionality reduction
Reinforcement Learning	Autonomous Driving	Self-driving cars learning optimal driving behavior
	Robotics	Training robots for automated assembly tasks
	Gaming	AI-driven strategy games like AlphaGo

Compiled for EEE Final Year by Manikanda Prabhu R

Choosing the Right Learning Approach

- **Supervised Learning:** When labeled data is available for prediction tasks like spam filtering, stock price forecasting.
- **Unsupervised Learning:** When exploring data structures without predefined labels like customer segmentation, anomaly detection.
- **Reinforcement Learning:** When decision-making is required in a dynamic environment like game AI, robotics, self-driving cars.

Perceptron Architecture

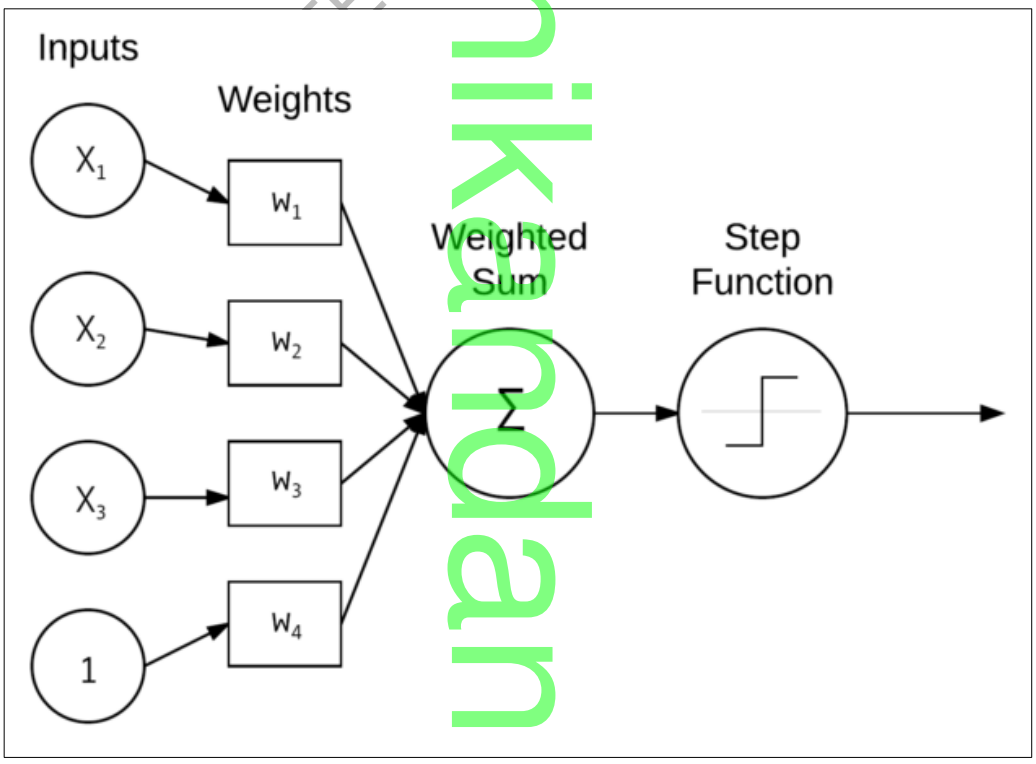
Rosenblatt (1958) defined a Perceptron as a system that learns using labeled examples (i.e., supervised learning) of feature vectors (or raw pixel intensities), mapping these inputs to their corresponding output class labels.

In its simplest form, a Perceptron contains *N* input nodes, one for each entry in the *input row* of the design matrix, followed by *only one layer* in the network with just a *single node* in that layer (Figure).

There exist connections and their corresponding weights $w_1, w_2, \ldots, w_i$ from the input x_i's to the single output node in the network. This node takes the weighted sum of inputs and applies a *step function* to determine the output class label. The Perceptron outputs either a 0 or a 1

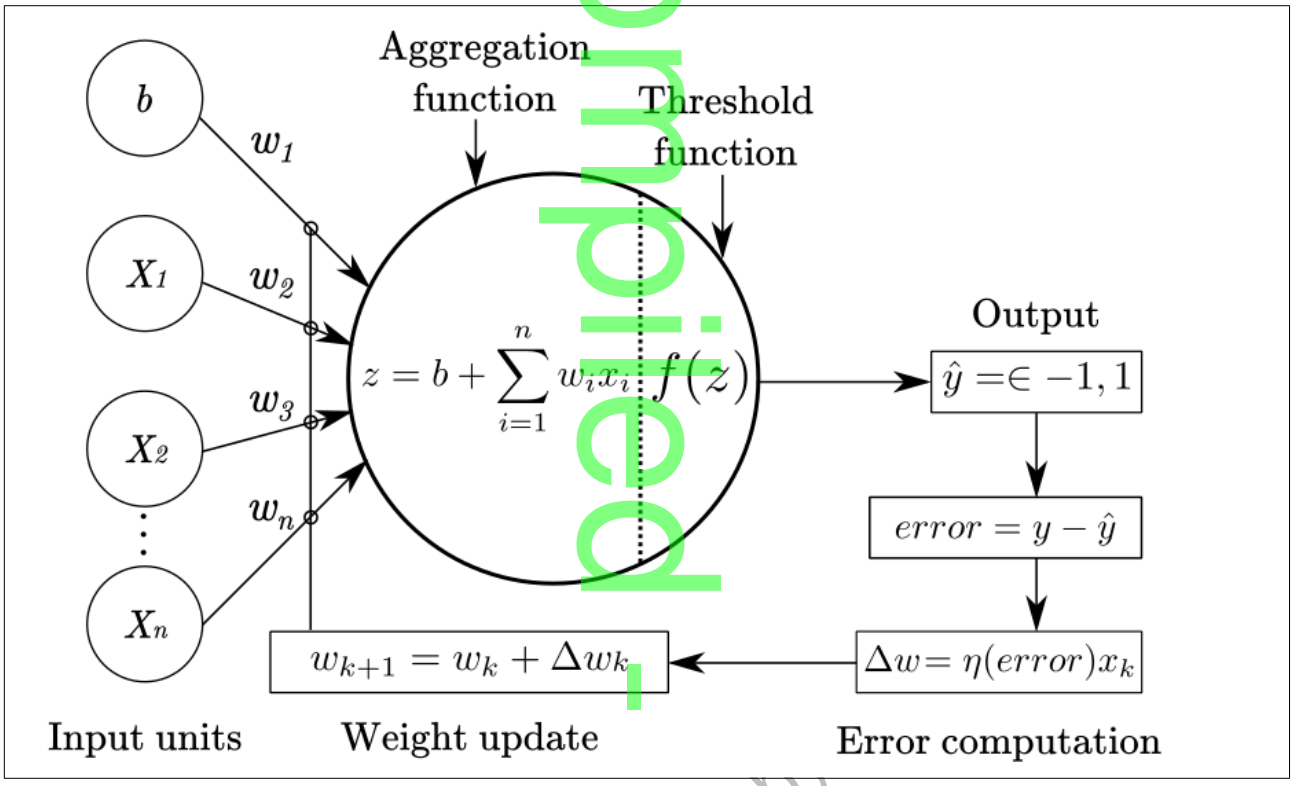
0 for class #1
and 1 for class #2;

thus, in its original form, the Perceptron is simply a binary, two-class classifier.

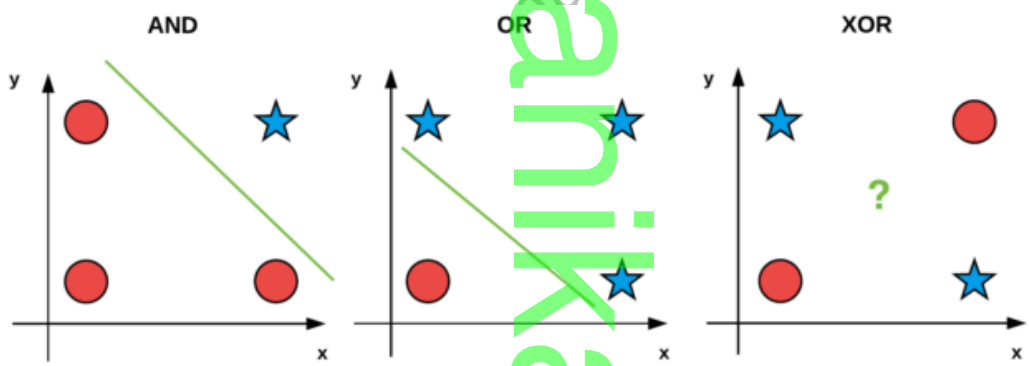


Compiled by Manikandan R
EEE Final Year by Manikandan P

Perceptron Learning Mechanism



Binary Classification



Workings of Perceptron Convergence Algorithm

Step-by -step working of this algorithm can be shown as:

1. Initialization: Start by setting the weights and bias to small random values. These weights determine the importance of each input feature.

2. Input: Feed an input data point (a vector of features) into the perceptron.

3.Weighted Sum: Calculate the weighted sum of the input features plus the bias. This is done by multiplying each input feature by its corresponding weight and adding them all up, along with the bias.
weighted sum = $(w_1 * x_1)+(w_2 * x_2)+\ldots+(w_n * x_n)+b$

Compiled - Manikandan R

4. Activation Function: Apply the activation function to the weighted sum to get the output. In a simple perceptron, this is usually a step function that outputs 1 if the weighted sum is positive, and -1 (or 0) if it's negative.

5. Prediction: Compare the perceptron's output (prediction) to the actual label of the input data point (either 1 or -1).

6. Update Weights: If the prediction is incorrect, adjust the weights and bias. The weights are updated to reduce the error, moving the decision boundary closer to correctly classifying the input. The update rule is:

- $w_i = w_i + \Delta w_i$
- $b = b + \Delta b$

where

- $\Delta w_i = \eta * (y - y') * x_i$
- $\Delta b = \eta * (y - y')$
- η is learning rate parameter
- y is the actual label
- y' is the predicted label

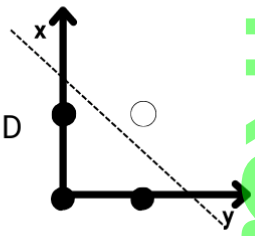
7**. Repeat:** Repeat steps 2-6 for all data points in the training set. This process is called an epoch. Multiple epochs are performed until the perceptron correctly classifies all training data points or a maximum number of epochs is reached.

8. **Convergence:** If the data is linearly separable, the algorithm will eventually find the correct weights and bias to perfectly classify the training data. This means the perceptron has "converged."

Output

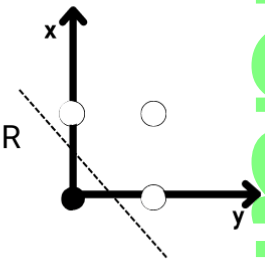
X1	X2	Y
F	F	0
F	T	0
T	F	0
T	T	1

AND



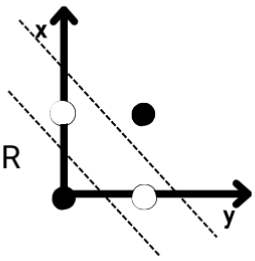
X1	X2	Y
F	F	0
F	T	1
T	F	1
T	T	1

OR



X1	X2	Y
F	F	0
F	T	1
T	F	1
T	T	0

XOR



enjoyalgorithms.com

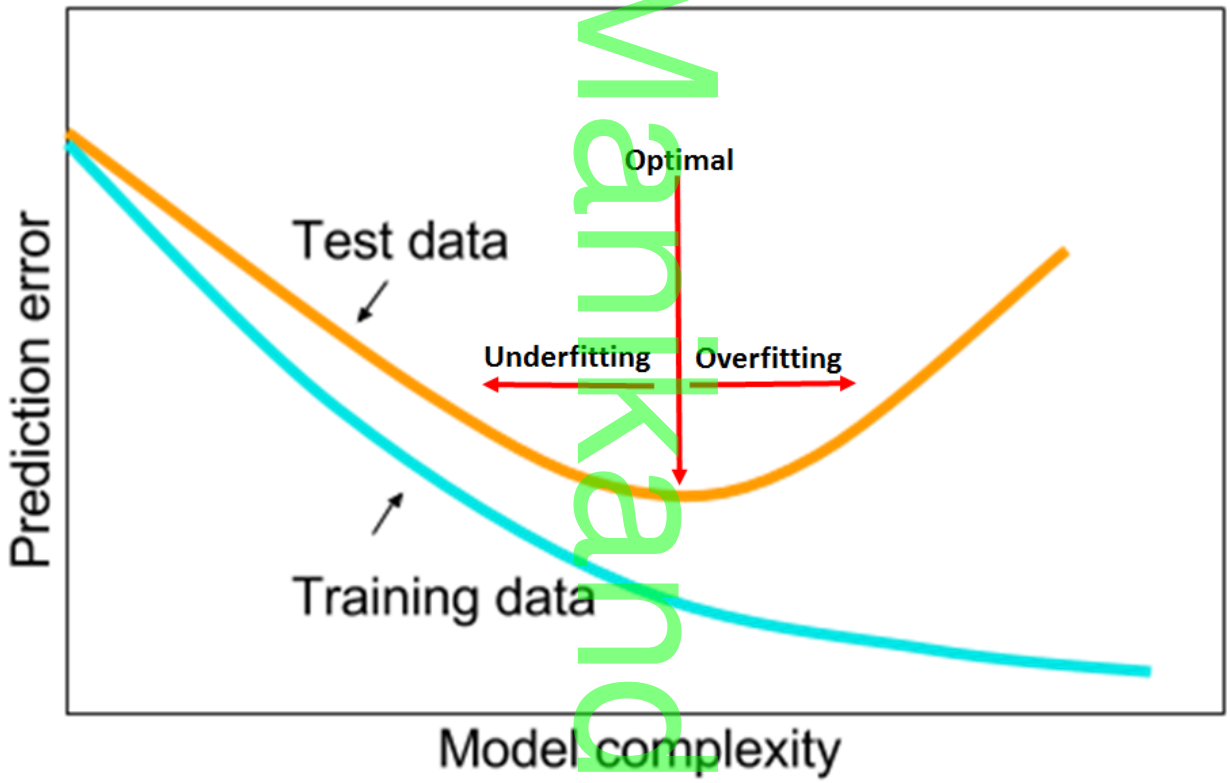
Compiled - Manikandan R
CE FinalYear by Manikandan R

MODEL TRAINING

Learning curve give insight into whether the model is under- or over-fitting

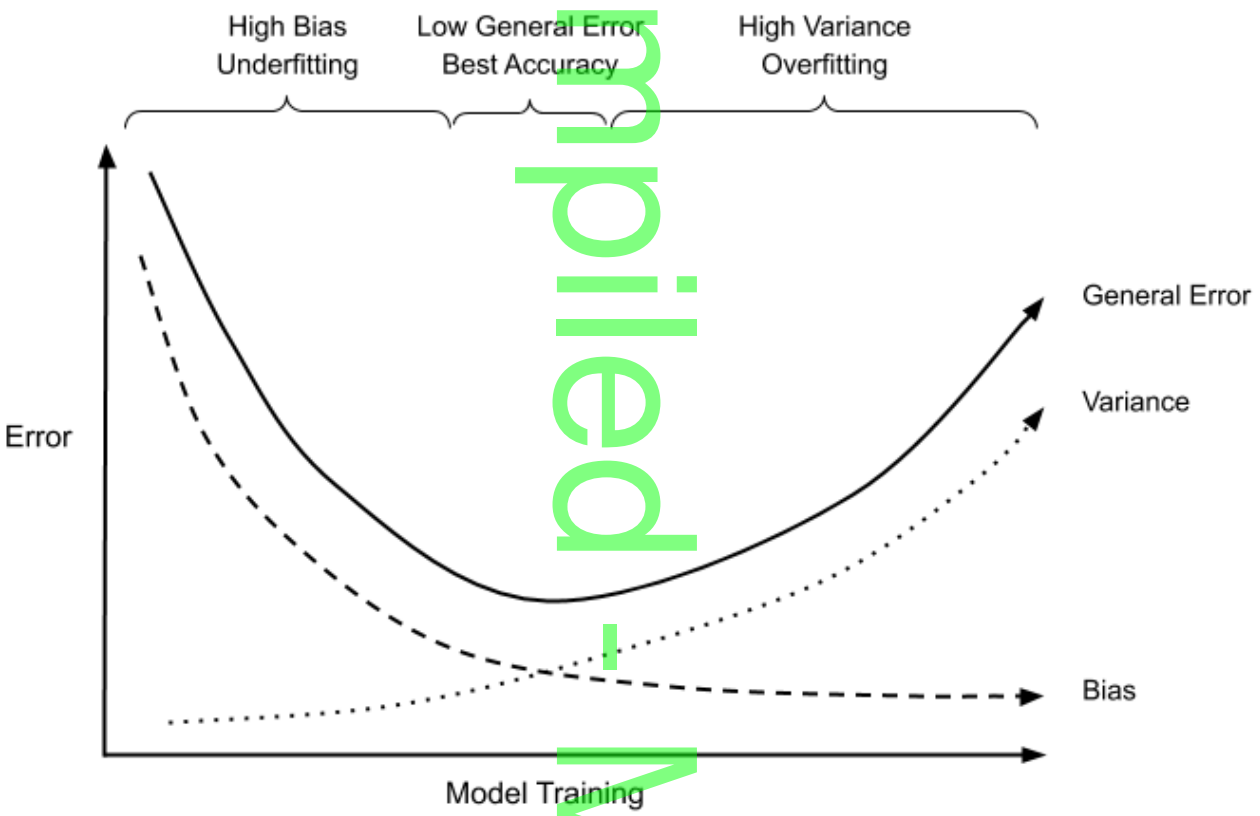
A learning curve is a diagnostic tool that plots the error metric used to evaluate a machine learning algorithm for both the training and validation data at each iteration of the algorithm. In most cases, the training error, or deviance, will continue to decrease as the model is built out, while the validation error decreases for a number of iterations before eventually increasing. The point at which the validation error first begins to rise provides guidance for an appropriate number of iterations to balance the bias/variance tradeoff. A classic learning curve is drawn below, with the optimal stopping point marked.

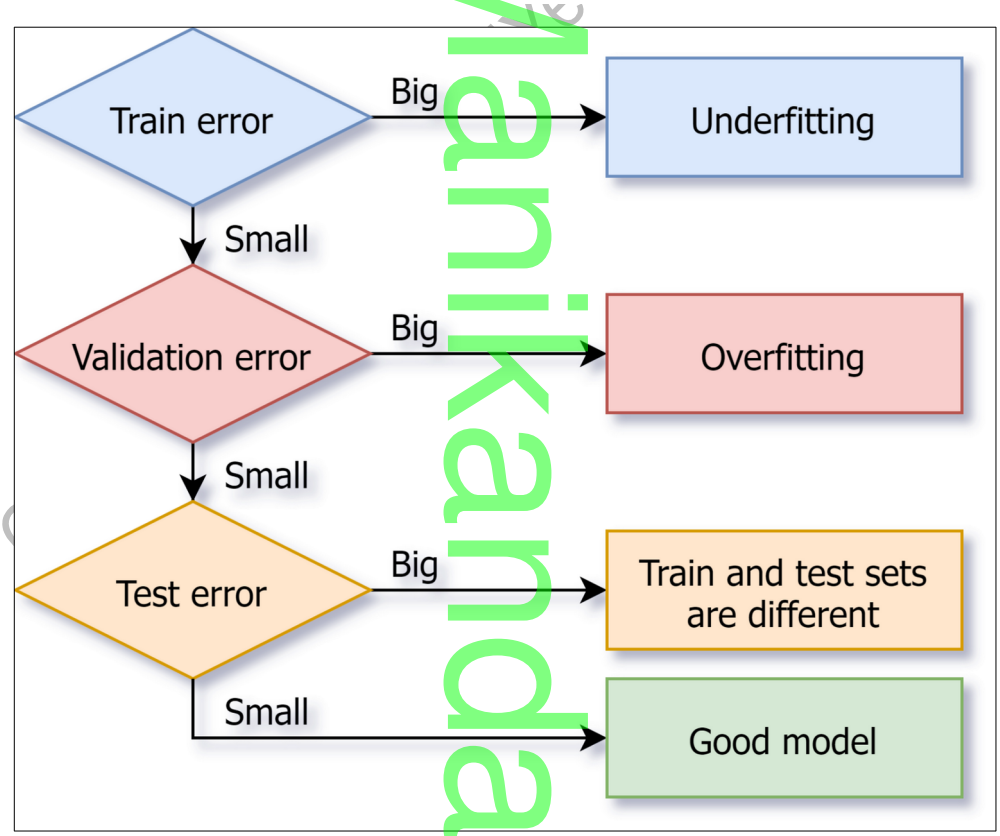
If a model is significantly underfit, both the training and validation error will be high and not significantly improve over further iterations. If the training error does not mostly flatten out by the last few iterations, it is likely a sign that the number of iterations are not sufficiently large enough for the algorithm to appropriately learn the data. On the other hand, if the training error is flat for many iterations while at the same time the validation error is increasing, the model is overfitting at that point of the algorithm, and the number of iterations should be decreased to the point at which the validation error first begins to



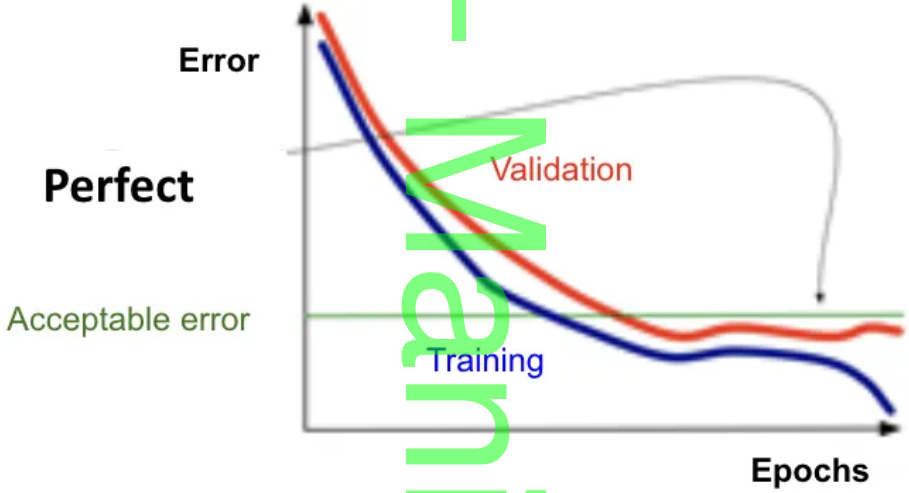
Compiled - Manikandan R
by Manikandan R

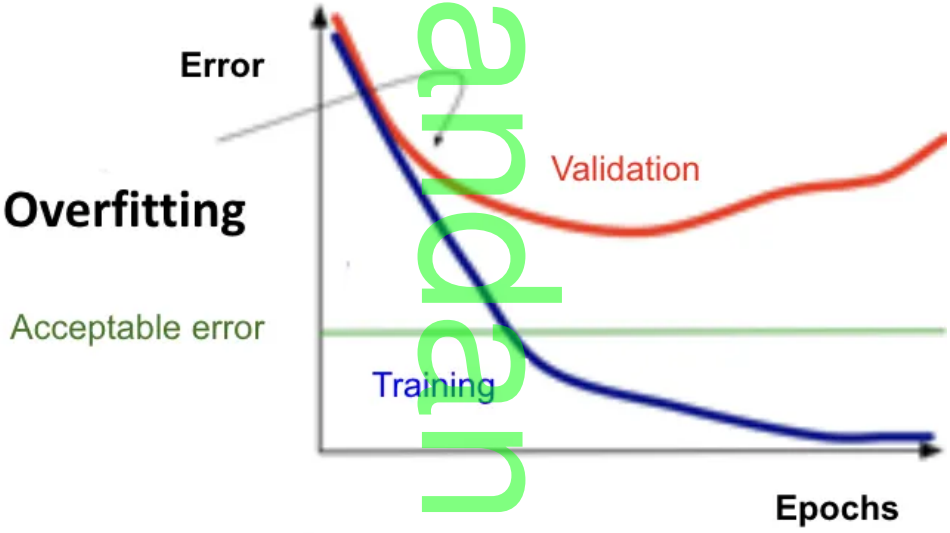
With Bias Variance





Compiled - Manikandan R





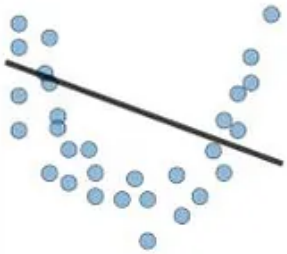
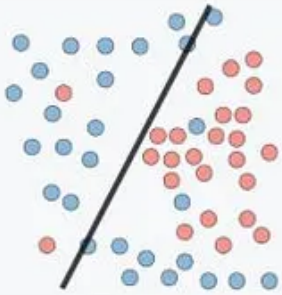
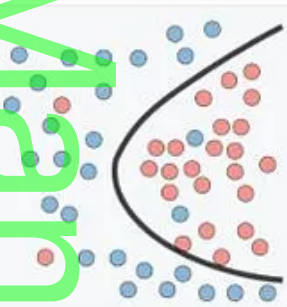
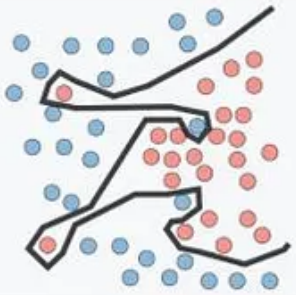
Compiled - Manikandan R

underfitting occurs when your model is **too simple** for our data.

- **overfitting** occurs when your model is **too complex** for our data.

Based on this, simple intuition we should keep in mind is:

- to fix **underfitting**, you should **complicate** the model.
- to fix **overfitting**, you should **simplify** the model.

	Underfitting	Just right	Overfitting
Symptoms	• High training error • Training error close to test error • High bias	• Training error slightly lower than test error	• Very low training error • Training error much lower than test error • High variance
Regression illustration			
Classification illustration			
Deep learning illustration			
Possible remedies	• Complexify model • Add more features • Train longer		• Perform regularization • Get more data

Compiled: Manu Kandan R

Techniques to fight underfitting and overfitting	
Underfitting	**Overfitting**
More complex model	More simple model
Less regularization	More regularization
Larger quantity of features	Smaller quantity of features
More data can't help	More data can help

Techniques to fight underfitting and overfitting		
	Underfitting	**Overfitting**
Complexity of the model	More complex model Try a more powerful model with a larger number of parameters Ensemble learning More layers / number of neurons per layer	More simple model Try a less powerful model with a fewer number of parameters Less layers / number of neurons per layer
Regularization	Less regularization Decrease regularization	More regularization Increase regularization impact Early stopping, L1 / L2 regularization, dropout
Quantity of features	A larger quantity of features Get additional features, feature engineering, polynomial features, etc.	A smaller quantity of features Remove all additional features, feature selection
Data	Data cleaning, hold-out validation or cross validation. ~~Getting more data most likely will not help~~	Data cleaning, hold-out validation or cross validation. Getting more data most likely will help (data augmentation)

Regularization

Regularization is an indirect and forced simplification of the model. The regularization term requires the model to keep parameter values as small as possible, so requires the model to be as simple as possible.

Compiled by Manikandan R

1. Supervised Models

Supervised learning is the study of algorithms that use labeled data in which each data instance has a known category or value to which it belongs. This results in the model to discover the relationship between the input features and the target outcome.

1.1 Classification

The classifier algorithms are designed to indicate whether a new data point belongs to one or another among several predefined classes. Imagine when you are organising emails into spam or inbox, categorising images as cat or dog, or predicting whether a loan applicant is a credible borrower. In the classification models, there is a learning process by the use of labeled examples from each category. In this process, they discover the correlations and relations within the data that help to distinguish class one from the other classes. After learning these patterns, the model is then capable of assigning these class labels to unseen data points.

Common Classification Algorithms:

- **Logistic Regression:** A very efficient technique for the classification problems of binary nature (two types, for example, spam/not spam).
- **Support Vector Machine (SVM):** Good for tasks like classification, especially when the data has a large number of features.
- **Decision Tree:** Constructs a decision tree having branches and proceeds to the class predictions through features.
- **Random Forest:** The model generates an "ensemble" of decision trees that ultimately raise the accuracy and avoid overfitting (meaning that the model performs great on the training data but lousily on unseen data).
- **K-Nearest Neighbors (KNN):** Assigns a label of the nearest neighbors for a given data point.

1.2 Regression

Regression algorithms are about forecasting of a continuous output variable using the input features as their basis. This value could be anything such as predicting real estate prices or stock market trends to anticipating customer churn (how likely customers stay) and sales forecasting. Regression models make the use of features to understand the relationship among the continuous features and the output variable. That is, they use the pattern that is learned to determine the value of the new data points.

Common Regression Algorithms

- **Linear Regression:** Fits depth of a line to the data to model for the relationship between features and the continuous output.
- **Polynomial Regression:** Similiar to linear regression but uses more complex polynomial functions such as quadratic, cubic, etc, for accommodating non-linear relationships of the data.
- **Decision Tree Regression:** Implements a decision tree-based algorithm that predicts a continuous output variable from a number of branching decisions.
- **Random Forest Regression:** Creates one from several decision trees to guarantee error-free and robust regression prediction results.
- **Support Vector Regression (SVR):** Adjusts the Support Vector Machine ideas for regression tasks, where we are trying to find one hyperplane that most closely reflects continuous output data.

Compiled for EEE Final Year by Manikandan R
Compiled - Manikandan R

2. Unsupervised Models

Unsupervised learning involves a difficult task of working with data which is not provided with pre-defined categories or label.

2.1 Clustering

Visualize being given a basket of fruits with no labels on them. The fruits clustering algorithms are to group them according to the inbuilt similarities. Techniques like K-means clustering are defined by exact number of clusters ("red fruits" and "green fruits") and then each data point (fruit) is assigned to the cluster with the highest similarity within based on features (color, size, texture). Contrary to this, hierarchical clustering features construction of hierarchy of clusters which makes it more easy to study the system of groups. Spatial clustering algorithm Density-Based Spatial Clustering of Applications with Noise (DBSCAN) detects groups of high-density data points, even in those areas where there is a lack of data or outliers.

2.2 Dimensionality Reduction

Sometimes it is difficult to both visualize and analyze the data when you have a large feature space (dimensions). The purpose of dimensionality reduction methods is to decrease the dimensions needed to maintain the key features. Dimensions of greatest importance are identified by principal component analysis (PCA), which is the reason why data is concentrated in fewer dimensions with the highest variations. This speeds up model training as well as offers a chance for more efficient visualization. LDA (Linear Discriminant Analysis) also resembles PCA but it is made for classification tasks where it concentrates on dimensions that can differentiate the present classes in the dataset.

2.3 Anomaly Detection

Unsupervised learning can also be applied to find those data points which greatly differ than the majorities. The statistics model may identify these outliers, or anomalies as signaling of errors, fraud or even something unusual. Local Outlier Factor (LOF) makes a comparison of a given data point's local density with those surrounding it. It then flags out the data points with significantly lower densities as outliers or potential anomalies. Isolation Forest is the one which uses different approach, which is to recursively isolate data points according to their features. Anomalies usually are simple to contemplate as they often necessitate fewer steps than an average normal point.

3. Semi-Supervised Model

Besides, supervised learning is such a kind of learning with labeled data that unsupervised learning, on the other hand, solves the task where there is no labeled data. Lastly, semi-supervised learning fills the gap between the two. It reveals the strengths of both approaches by training using data sets labeled along with unlabeled one. This is especially the case when labeled data might be sparse or prohibitively expensive to acquire, while unlabeled data is undoubtedly available in abundance.

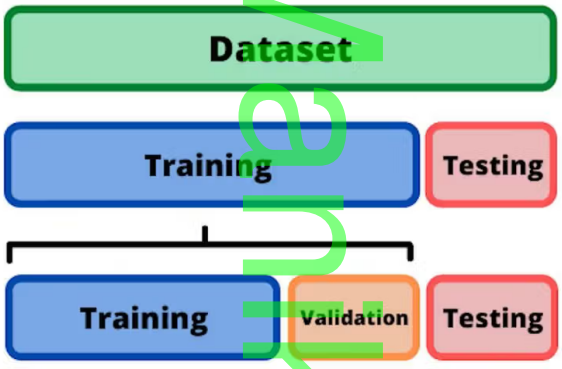
Modeling ML Algorithm/How Machine Learning Works?

1. **Model Represntation:** Machine Learning Models are represented by mathematical functions that map input data to output predictions. These functions can take various forms, such as linear equations, decision trees , or complex neural networks.
2. **Learning Algorithm:** The learning algorithm is the main part of behind the model's ability to learn from data. It adjusts the parameters of the model's mathematical function iteratively during

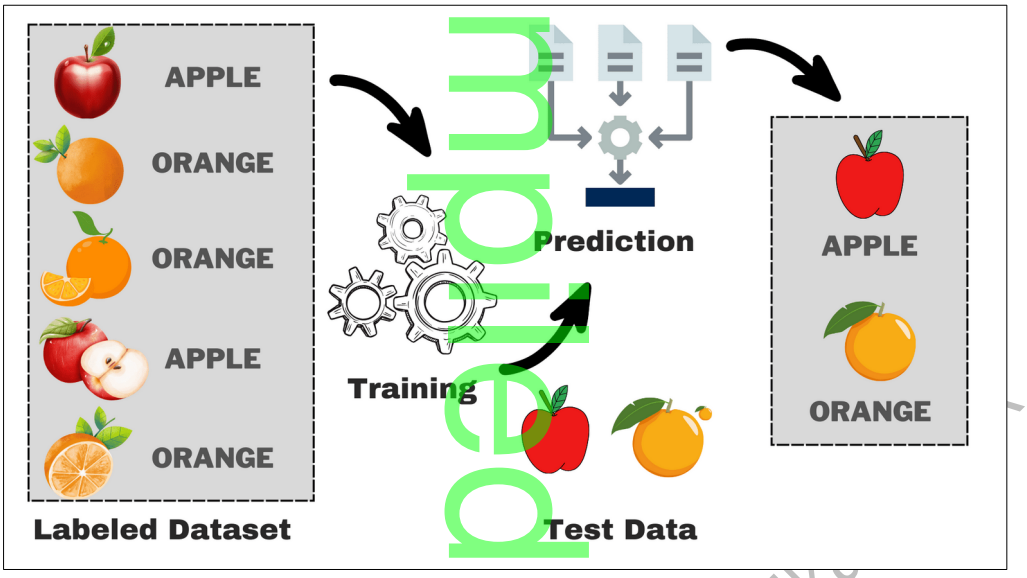
Compiled for EEE Final Year by Marikandan R
Compiled by Marikandan R

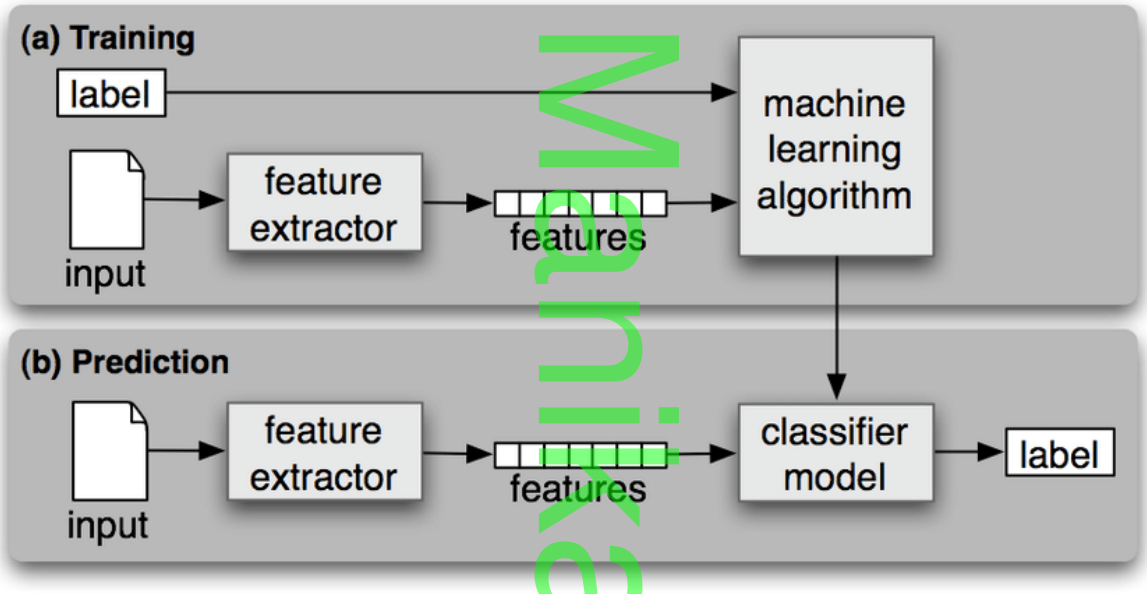
the training phase to minimize the difference between the model's prediction and the actual outcomes in the training data .

3. **Training Data:** Training data is used to teach the model to make accurate predictions. It consists of input features(e.g variables, attributes) and corresponding output labels(in supervised learning) or is unalabeled(in supervised learning). During training , the model analyzes the patterns in the training data to update its parameters accordingly.
4. **Objective Function:** The objective function, also known as the loss function, measures the difference between the model's predictions and the actual outcomes in the training data. The goal during training is to minimize this function, effectively reducing the errors in the model's predictions.
5. **Optimization Process:** Optimization is the process of finding the set of model parameters that minimize the objective function. This is typically achieved using optimization algorithms such as gradient descent, which iteratively adjusts the model's parameters in the direction that reduces the objective function.
6. **Generalization:** Once the model is trained, it is evaluated on a separate set of data called the validation or test set to assess its performance on new, unseen data. The model's ability to perform well on data it hasn't seen before is known as generalization.
7. **Final Output:** After training and validation, the model can be used to make predictions or decisions on new, unseen data. This process, known as inference, involves applying the trained model to new input data to generate predictions or classifications.



Compiled by Manikandan R





Compiled - Manikandan R